

目录



- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

Search

原创

web-access Skill上线：PaiCLI 的联网能力拉到满中满

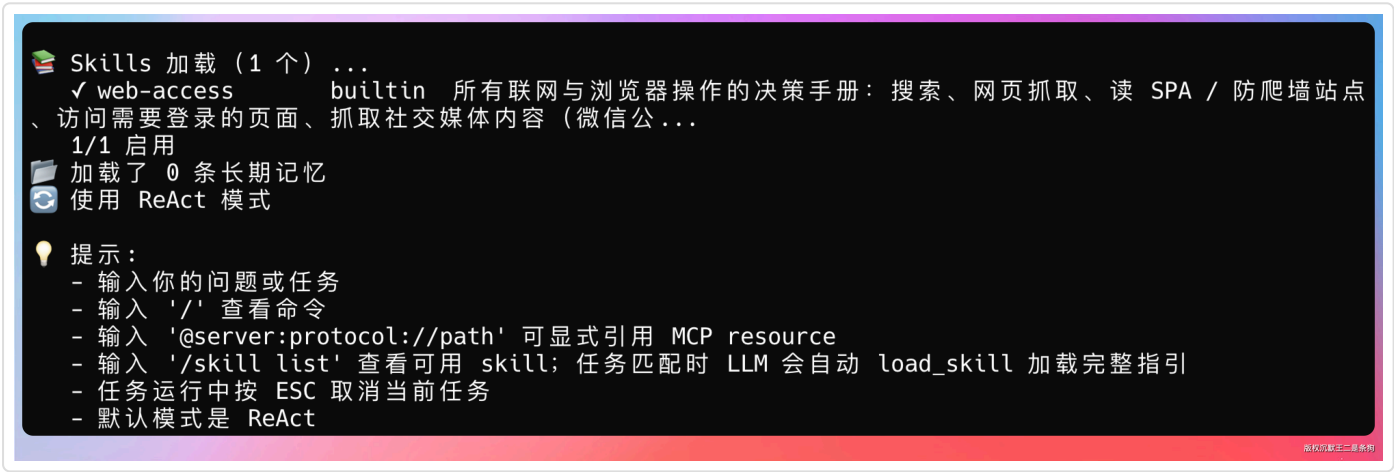
大家好，我是二哥呀。

上一期我们搞定了 CDP 会话复用，Agent 终于能直连日常 Chrome 了，GitHub 私有仓库、内网系统、飞书文档这些需要登录的内容统统能让Agent看到了。

但又有了新的问题。

比如说，让 PaiCLI 抓一篇未知的 URL。

合理的决策是：先用 web_fetch 试试能不能直接抓到正文，抓不到就切 Chrome DevTools MCP 上浏览器，浏览器也抓不到就走 Jina Reader 兜底。



这一期，我们给 PaiCLI 加上 Skill 系统。

决定 Agent 在什么场景下用什么工具、遇到阻拦怎么绕。加完之后，PaiCLI 就从一个“有一堆工具的 Agent”变成了一个“有经验的 Agent”。

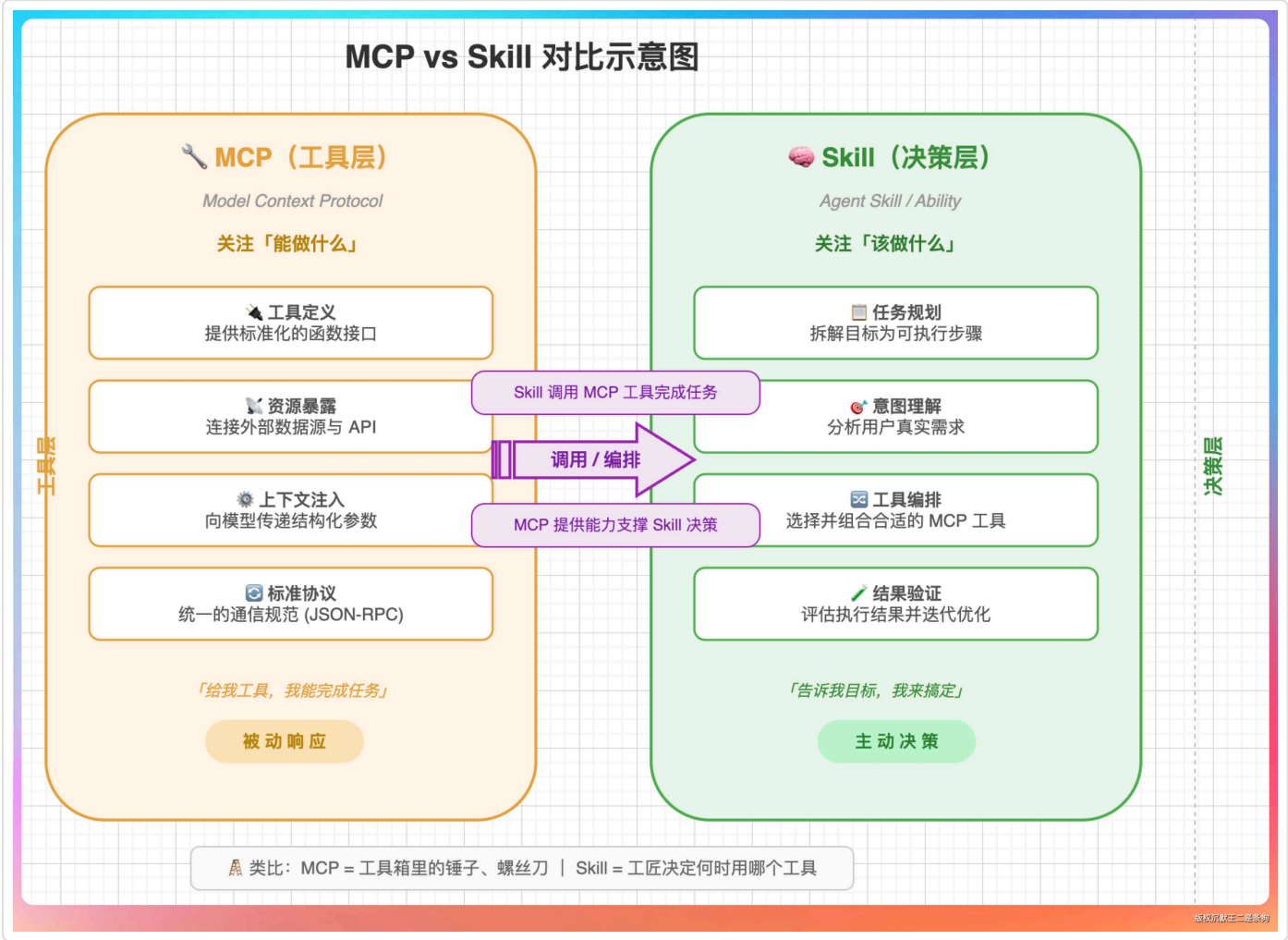
01、Skill 和 MCP 的区别

MCP 提供的是能力，能搜索、能抓网页、能操作浏览器。

Skill 提供的是决策，什么时候搜索、什么时候抓网页、什么时候启动浏览器。

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)



目录

- 01、Skill 和 MCP 的区别
- 02、Skill的三层加载架构
- 03、SKILL.md 的结构
- 04、手写 YAML 解析器
- 05、让 LLM 自己决定加载什么
- 06、user message 的精妙设计
- 07、SkillContextBuffer 的生...
- 08、web-access Skill 深度解...
- 09、/skill 命令组实操
- 10、写一个自己的 Skill
- 11、PaiCLI如何写到简历上？

Claude Code 在 2025 年底首先引入了 Skill 的概念。

一个 Skill 就是一个文件夹，里面放一个 **SKILL.md**（决策手册）加上可选的 references 参考文件。

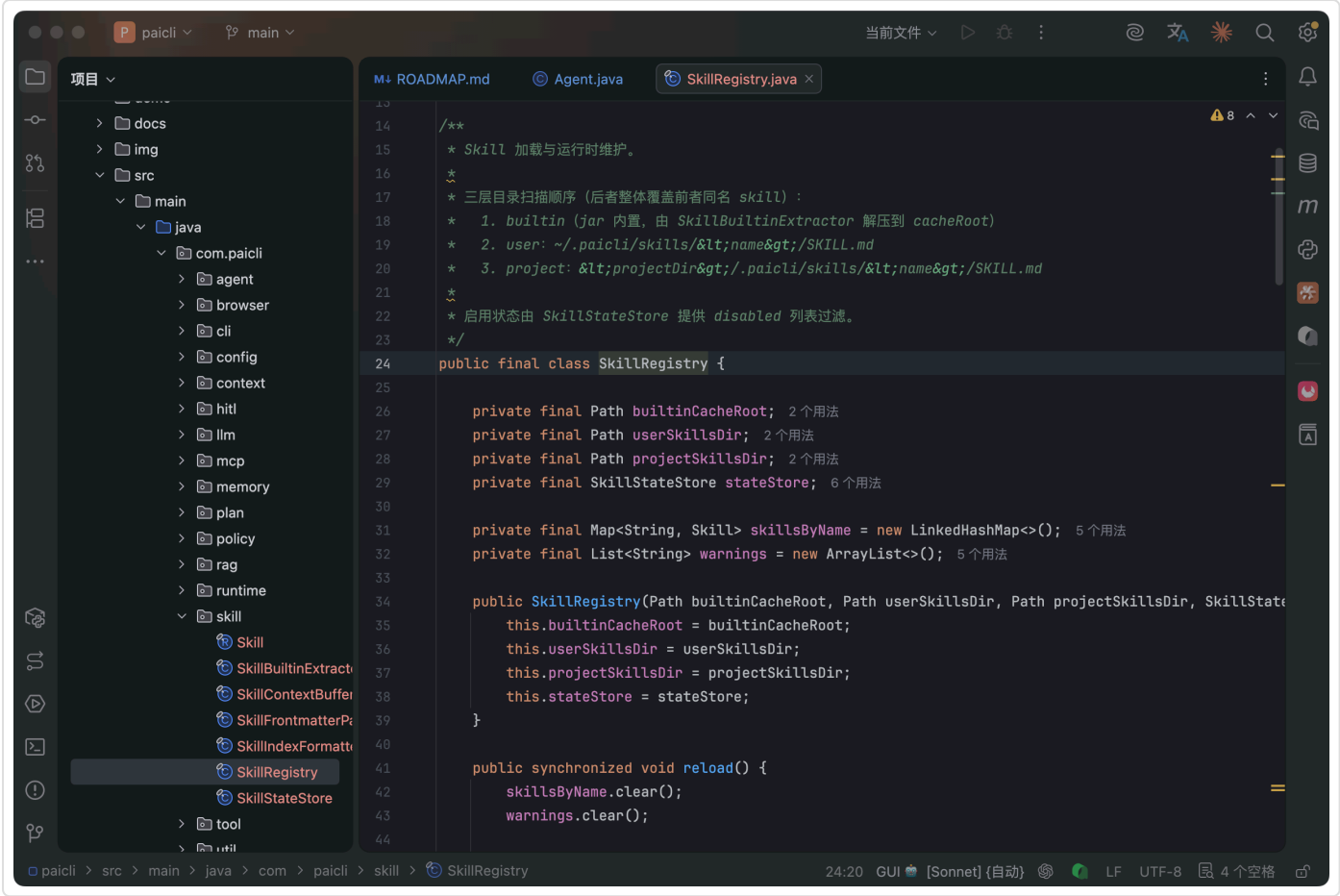
截止到目前，SKILL.md 已经不是 Claude Code 一家的事了。Anthropic、OpenAI、Google 三家在 Linux Foundation 下面共同成立了 Agentic AI Foundation，把 SKILL.md 定成了跨工具的开放标准。

也就是说，你给 Claude Code 写的 Skill，拿到 Codex 上也能直接用。

02、Skill的三层加载架构

PaiCLI 的 Skill 系统设计了三层加载机制：

- 第一层：内置 Skill**，打包在 PaiCLI 的 jar 包里，随版本发布。目前内置了一个 **web-access** Skill，教 Agent 怎么做联网操作。
- 第二层：用户级 Skill**，放在 `~/.paicli/skills/<name>/SKILL.md`。放自己写的全局 Skill，所有项目都能用。
- 第三层：项目级 Skill**，放在 `<项目目录>/.paicli/skills/<name>/SKILL.md`。针对特定项目的 Skill，优先级最高。



SkillRegistry 是管理这三层扫描和合并的核心类。扫描的时候按 builtin → user → project 的顺序处理，每扫到一个同名 Skill 就直接覆盖前一层的。

PaiCLI 启动时会输出一段 Skill 加载汇总：

 Skills 加载 (1 个) ...

✓ web-access

builtin

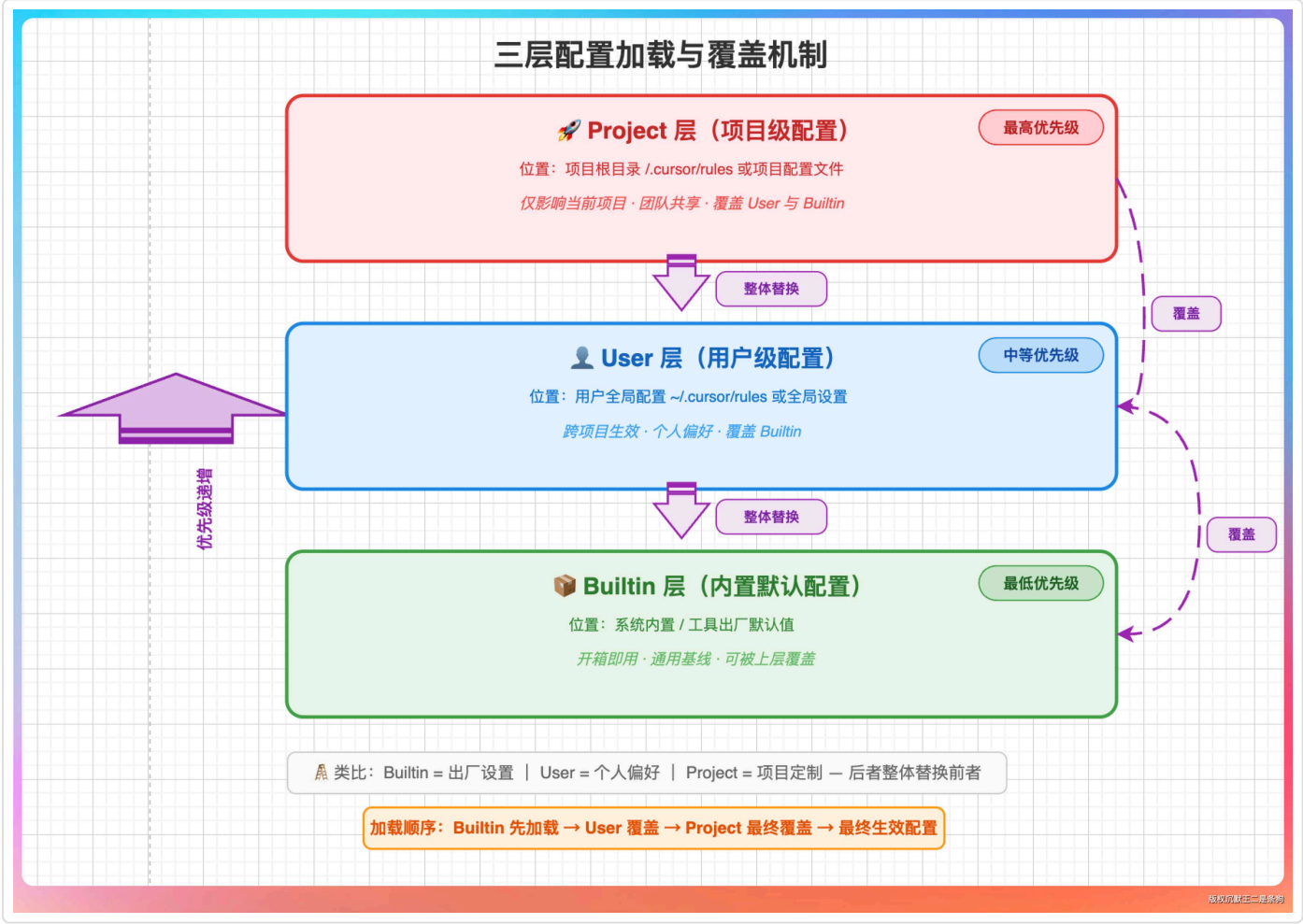
description 88 字符

1/1 启用, 索引段共

0.6KB

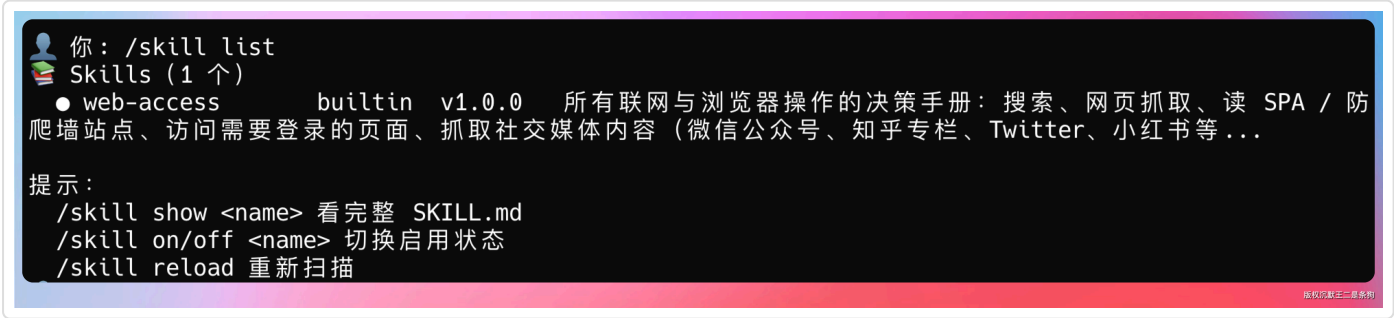
bash 复制代码

这段日志一眼就能看到有多少 Skill 被加载了、来源是什么、索引段占了多大空间。



来验证一下。

启动 PaiCLI，输入 `/skill list`：



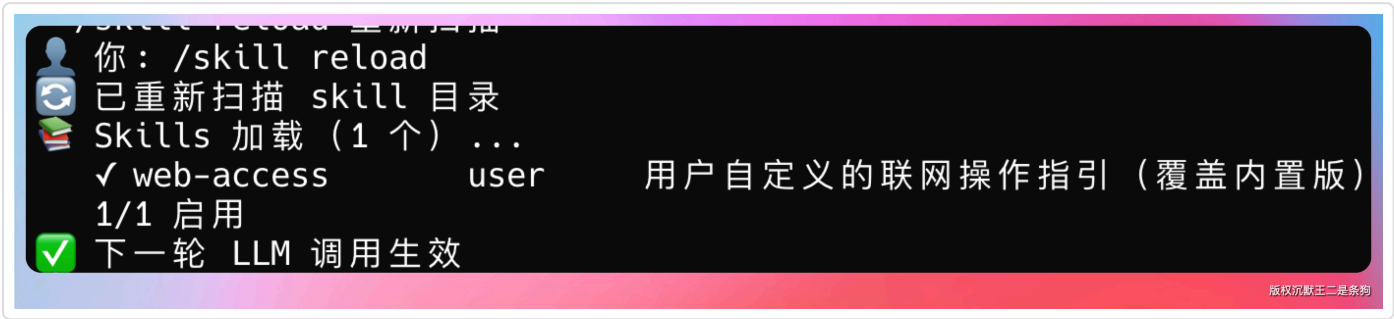
只有一个内置的 web-access。现在我在用户目录创建一个同名 Skill 试试覆盖效果：

bash 复制代码

```
mkdir -p ~/.paicli/skills/web-access
cat > ~/.paicli/skills/web-access/SKILL.md << 'EOF'
---
name: web-access
description: 用户自定义的联网操作指引（覆盖内置版）
version: "9.9.9"
---

这是用户版的 web-access，优先级高于内置版。
EOF
```

然后在 PaiCLI 里执行 `/skill reload`：



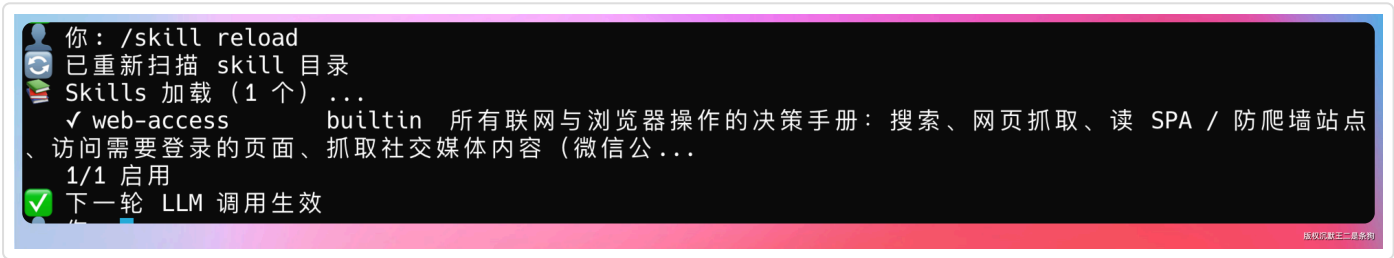
来源从 `builtin` 变成了 `user`，覆盖生效了。

验证完记得把用户级的删掉，恢复内置版本：

bash 复制代码

```
rm -rf ~/.paicli/skills/web-access
```

再 `/skill reload` 就回到内置的了。



03、SKILL.md 的结构

每个 Skill 的核心就是一个 `SKILL.md` 文件，分两部分：YAML frontmatter（元数据）和 Markdown body（决策手册正文）。

markdown 复制代码

```
---
name: web-access
description: |
  所有联网操作必须通过此 skill 处理，
  包括搜索、网页抓取、登录后操作
version: "1.0.0"
author: PaiCLI
tags: [web, browser, search]
---

# web-access Skill

## 浏览哲学

明确目标 → 选择起点 → 过程校验 → 完成判断...
```

frontmatter 只有两个必填字段：`name` 和 `description`。

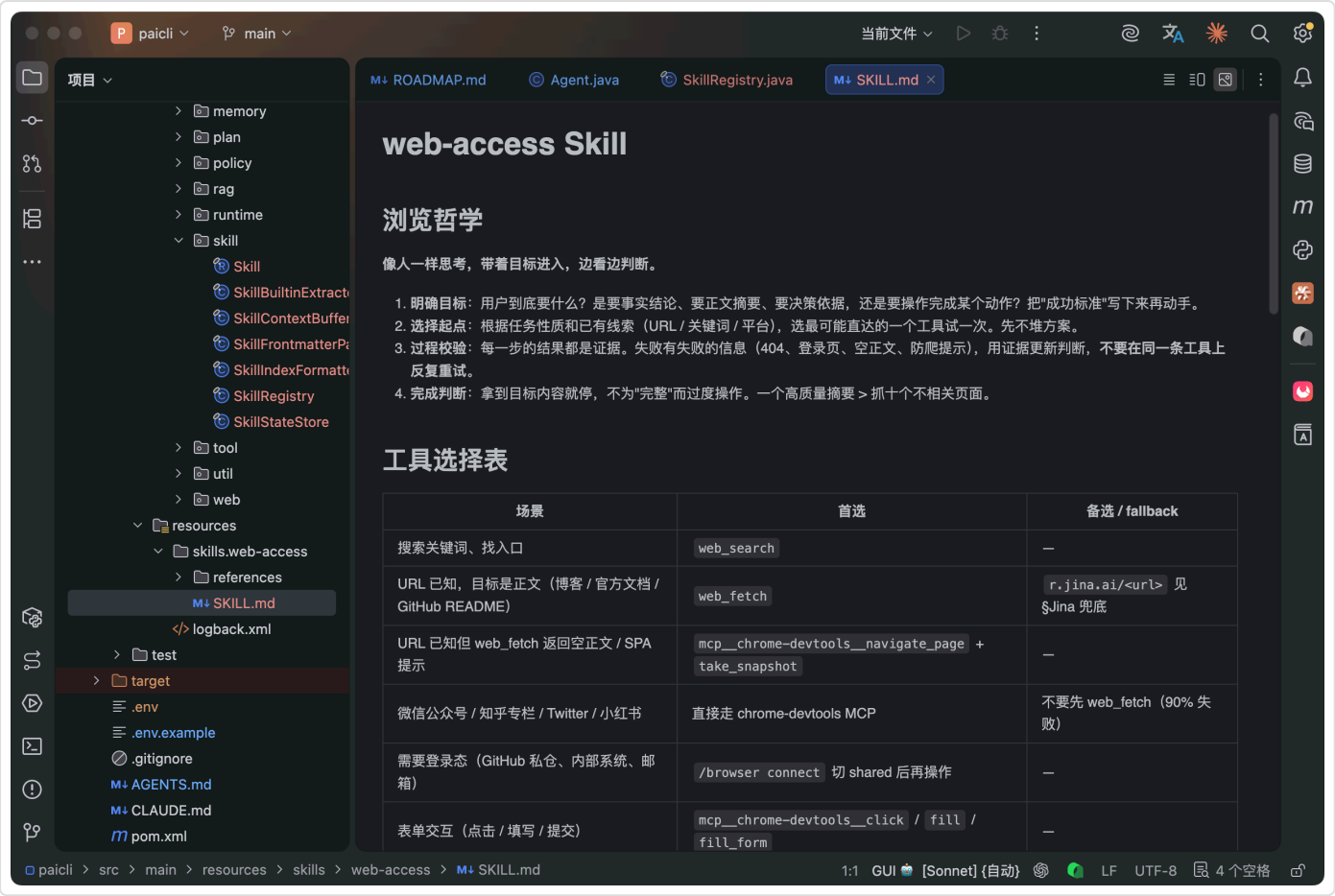
`version`、`author`、`tags` 都是选填。未知字段直接忽略。

body 部分就是给 LLM 看的决策手册。写什么都行，但核心是告诉 LLM **遇到什么场景做什么决策**。

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

这里有个关键设计：body 不是启动时就塞进 system prompt 的。LLM 在 system prompt 里只能看到每个 Skill 的 name 和 description（一句话摘要），需要时才通过 `load_skill` 工具加载完整 body。

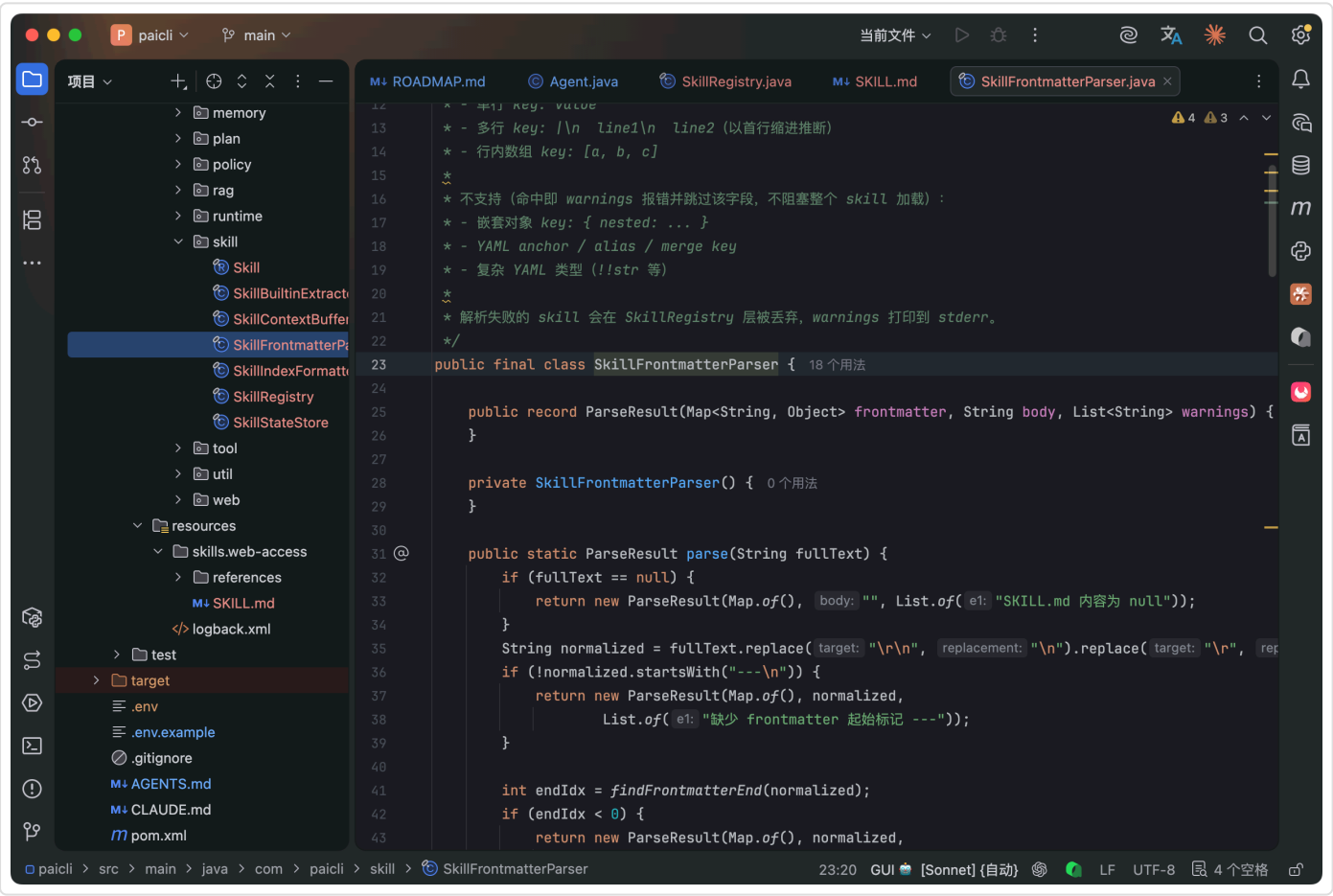


为什么这么设计？

因为 system prompt 是有 token 预算的。如果启动时把所有 Skill 的完整内容都塞进去，装 10 个 Skill 就可能吃掉好几万 token。按需加载，轻量索引，这是 Claude Code 的 Skill 系统采用的设计理念，叫做 **Progressive Disclosure（渐进式披露）**。

04、手写 YAML 解析器

`SkillFrontmatterParser` 覆盖了 95% 的真实使用场景。



来看几个 case。正常的单行值：

```
name: web-access
version: "1.0.0"
```

yaml 复制代码

多行 description（`|` 管道符）：

yaml 复制代码

```
description: |
  所有联网操作必须通过此 skill 处理，
  包括搜索、网页抓取、登录后操作
```

内联数组：

yaml 复制代码

```
tags: [web, browser, search]
```

如果写了我们不支持的语法，比如嵌套对象 `{nested: object}`，解析器会跳过这个字段并在 `stderr` 输出一条警告，不会阻塞其他 Skill 的加载。

bash 复制代码

```
⚠ Skill 'broken' frontmatter 解析警告：第 5 行包含不支持的语法，已跳过
```

05、让 LLM 自己决定加载什么

这是整个 Skill 系统最核心的机制。

传统做法是用关键词匹配。用户说“帮我看网页”，就自动加载 `web-access Skill`。但关键词匹配永远不够精确，“看网页”“浏览器”“抓取”“搜索”都可能触发，也可能漏掉。

PaiCLI 的做法是：把 `load_skill` 注册为一个内置工具，让 LLM 自己判断要不要调用。

LLM 的 `system prompt` 里会有一段 Skill 索引：

markdown 复制代码

```
## 可用 Skills (按需调用 load_skill 加载完整指引)

- **web-access**: 所有联网操作必须通过此 skill 处理，包括搜索、网页抓取、登录后操作...

判断准则：当任务描述匹配某个 skill 的触发场景时，调用 load_skill(name) 加载完整指引，
然后按指引执行。已加载的 skill 会在下一轮以 `## 已加载 Skill` 段落出现在你的 user message 中。
不要重复加载同一 skill；同一会话内一次足够。
```

LLM 看到问题涉及联网操作，就自己调 `load_skill("web-access")`。

来验证一下。直接对 PaiCLI 说：

shell 复制代码

```
> 帮我看下 https://mp.weixin.qq.com/s/RB7kF_BbsJZ5_Hmu9PxWdg 这篇文章讲了什么
```

观察 Agent 的行为：



目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

LLM 没有直接冲上去抓网页，而是先加载了 web-access 的决策手册。然后在下一轮，它按照决策手册的指引，先用 web_fetch 试了一次（微信文章是 SPA，抓不到正文），接着切 Chrome DevTools MCP 用浏览器打开页面拿到了完整内容。

这就是 Skill 的价值——让 Agent 学会正确的做事方法。

06、user message 的精妙设计

这个设计细节是整个 Skill 系统里最值得深挖的点。

当 LLM 调用 load_skill("web-access") 时，PaiCLI 做了两件事：

- 工具返回一条简短确认：“已加载 skill 'web-access' 的完整指引（3.2KB），将在下一轮上下文中体现”
- 把 SKILL.md 的 body 写入 SkillContextBuffer

注意，工具返回的结果里没有包含 body 的完整内容。body 是在下一轮构造 user message 时，从 buffer 里取出来拼到用户输入的前面：

yaml 复制代码

```
## 已加载 Skill: web-access
<SKILL.md body 完整内容>

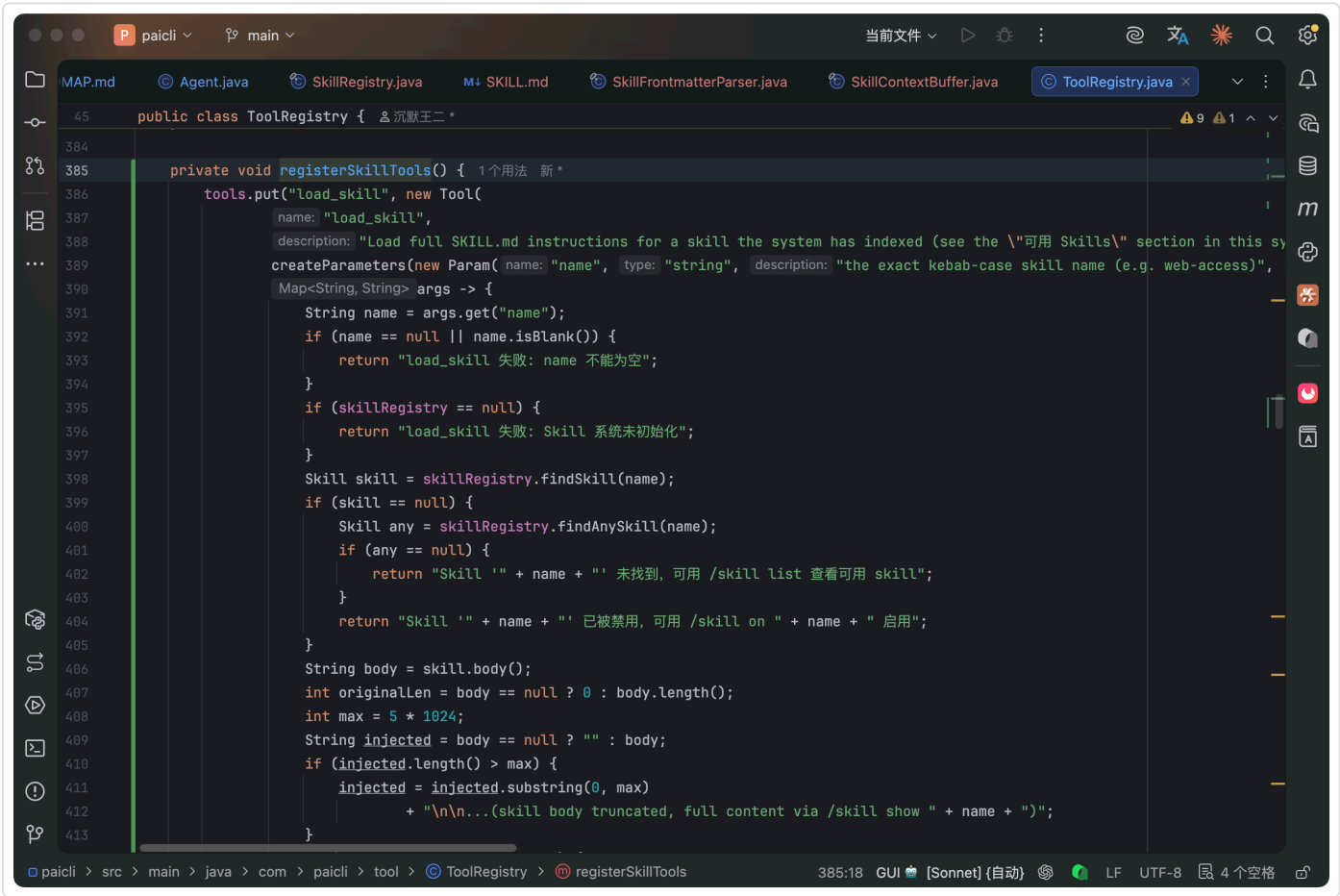
---
用户输入: <用户的原始消息>
```

为什么不直接在工具返回结果里塞 body？

为什么不塞进 system prompt？

第一个问题：工具返回的内容在 LLM 眼里是“事实输入”，LLM 倾向于把它当做参考信息。但 SKILL.md 的 body 是“操作指引”，我们希望 LLM 把它当做指令来执行。放在 user message 里，LLM 会把它当成“用户附加要求”，决策权重更高。

第二个问题：system prompt 一旦改变，API 的 prompt cache 就会失效。如果每次 load_skill 都去改 system prompt，之前缓存的几千个 token 全部作废。走 user message 注入，system prompt 始终不变，prompt cache 得以保留。



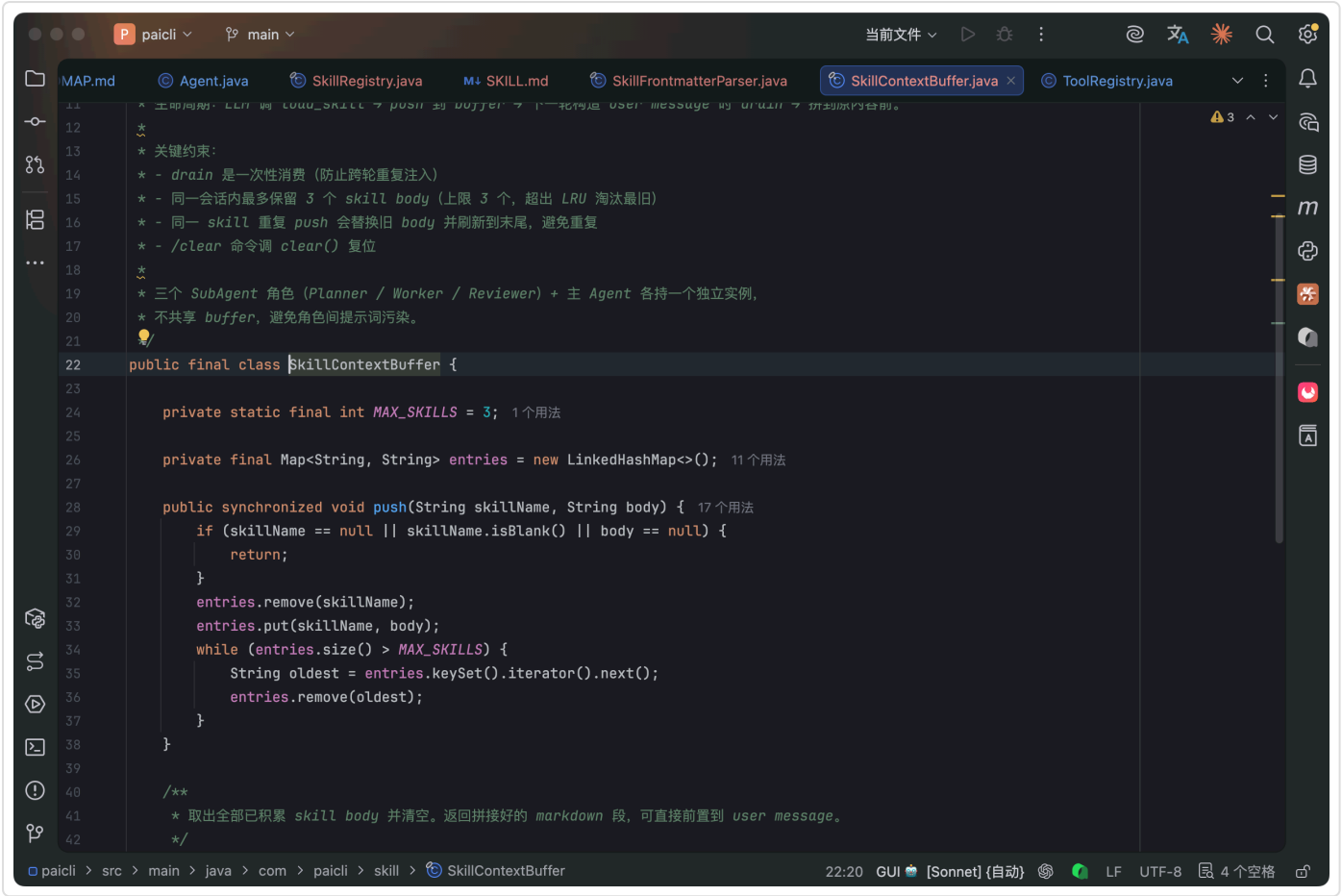
从实现角度看，load_skill 的代码在 ToolRegistry.registerSkillTools() 里。先从 SkillRegistry 查 skill 是否存在且启用，然后读 body 内容，截断到 5KB，push 进 SkillContextBuffer，最后返回一条确认消息。整个流程非常干净，没有任何副作用。

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

07、SkillContextBuffer 的生命周期

SkillContextBuffer 是整个注入机制的核心数据结构。



它的生命周期有几个关键特性：

- ①、**一次性消费**：drain() 取出内容后 buffer 清空。下一轮 user message 不会再携带上一轮已注入的 Skill body。这避免了 body 在对话中反复累积撑爆上下文。
- ②、**最多 3 个 Skill**：如果 LLM 在同一轮连续调了 3 个以上的 load_skill，buffer 只保留最近的 3 个。
- ③、**同名替换**：同一个 Skill 被加载两次，新的 body 替换旧的，不会重复累积。
- ④、**角色隔离**：在 PlanExecute 模式下，Planner、Worker、Reviewer 三个角色各自持有独立的 buffer 实例，互不干扰。AgentOrchestrator 在创建 SubAgent 时为每个角色分配独立的 SkillContextBuffer。

为什么要隔离？

因为 Worker 可能加载了 web-access 去抓网页，而 Reviewer 不需要这个 Skill 的决策指引——它的职责是审查代码质量，不是浏览网页。如果共享 buffer，Reviewer 的 user message 里会被塞入一堆不相关的浏览指引，白白浪费 token。

- ⑤、**/clear 重置**：执行 /clear 命令会清空 buffer，下一轮从零开始。这在调试 Skill 的时候特别有用。改了 SKILL.md 的内容后，先 /clear 清掉旧的 buffer，再 /skill reload 重新加载，保证 Agent 读到的是最新版本。

先让 Agent 加载 web-access：

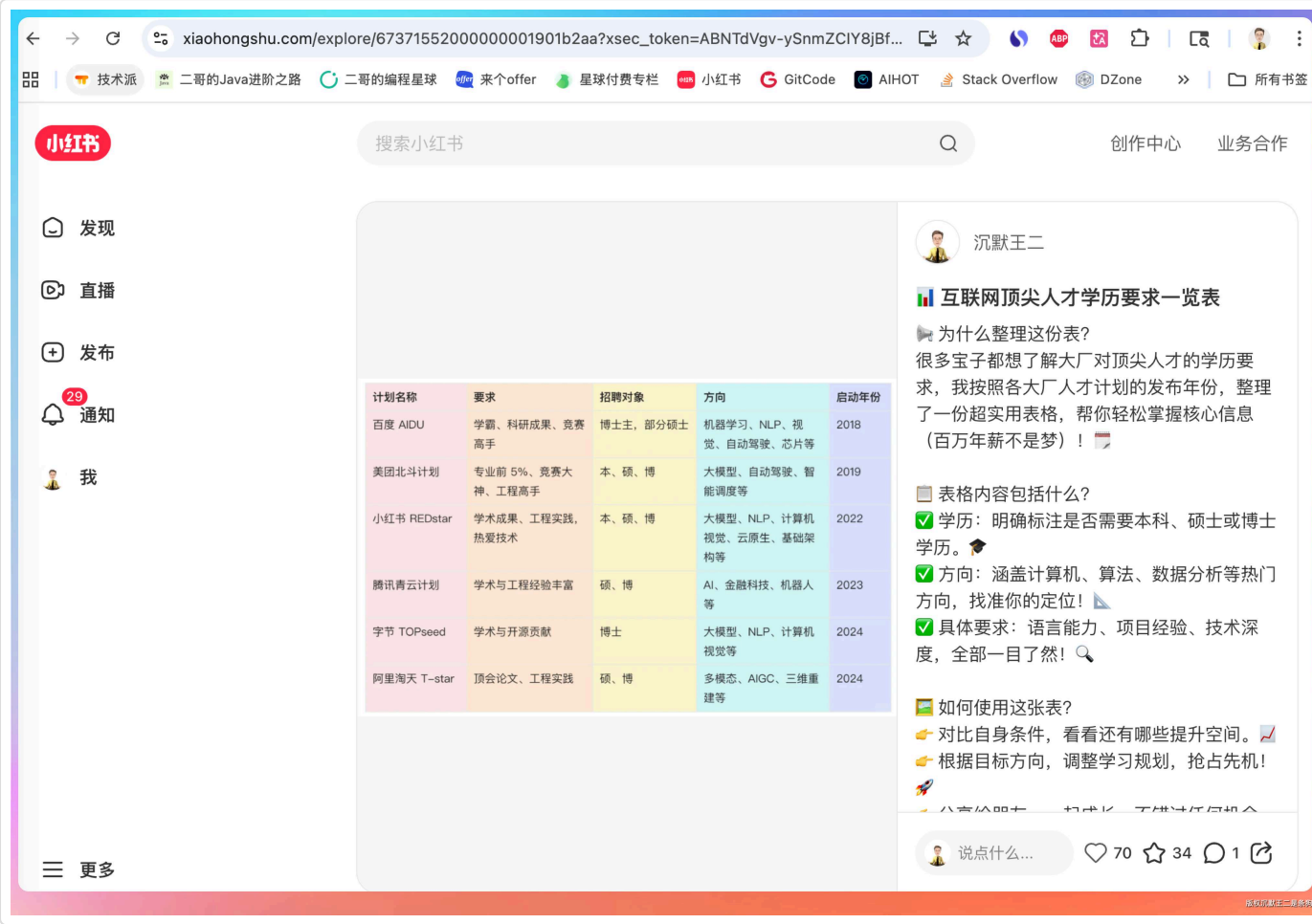
less 复制代码

```
> 帮我看 https://mp.weixin.qq.com/s/RB7kF_BbsJZ5_Hmu9PxWdg
[Agent 调用 load_skill("web-access"), 完成操作]

> 再看一篇 https://www.xiaohongshu.com/explore/67371552000000001901b2aa?
xsec_token=ABNTdVgv-ySnmZCIY8jBfaLyQ4YqdGukYbpdR_-S6j-0=&xsec_source=pc_user
[观察：Agent 不会重复调用 load_skill，因为 system prompt 提示了“同一会话内一次足够”]
```

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)



第二轮 user message 里不会再出现 ## 已加载 Skill: web-access 段落，但 LLM 记得上一轮已经看过决策手册的内容，继续按指引行动。

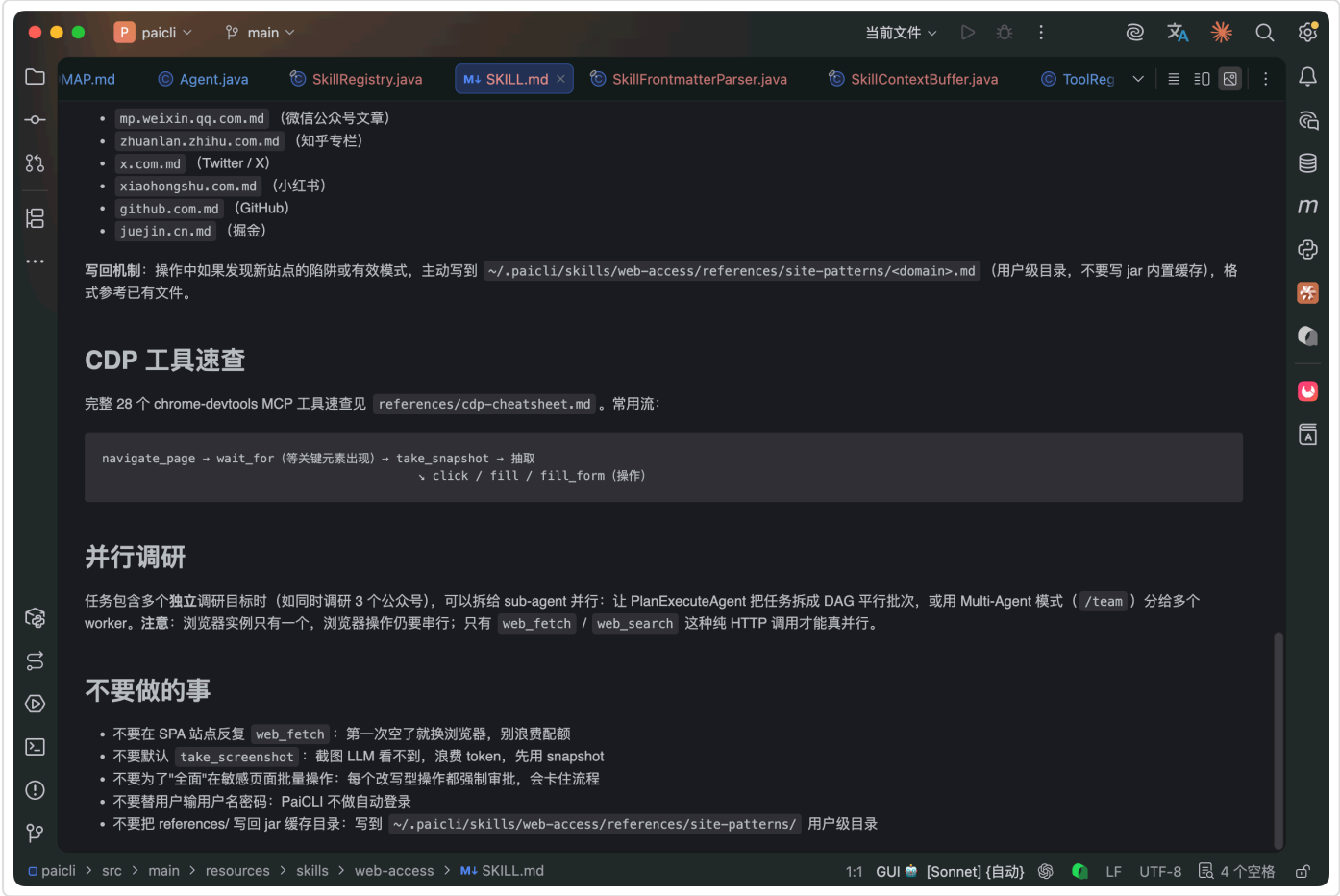
08、web-access Skill 深度解析

PaiCLI 内置的第一个 Skill 就是 web-access，是使用频率最高的决策手册。

上一期我们已经讲过 CDP 的原理，这一期重点看 web-access 作为 Skill 给 Agent 带来了什么决策能力。

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)



web-access 的 SKILL.md 大致分这几个板块：

- ①、**浏览哲学**，四步法则：明确目标（要拿什么信息）→ 选择起点（用最轻量的方式尝试）→ 过程校验（拿到的内容是否符合预期）→ 完成判断（信息是否充分）。
- ②、**工具选择表**，不同场景对应不同工具。搜索用 `web_search`，已知 URL 用 `web_fetch`，SPA 动态渲染站点用 Chrome DevTools MCP 的 `navigate_page` + `take_snapshot`，`web_fetch` 和浏览器都搞不定的用 Jina Reader (`curl https://r.jina.ai/<url>`) 兜底。
- ③、**浏览器优先级**，这是决策手册最精华的部分。渐进式升级策略：先 `web_fetch` 试一把（成本最低，token 最少）→ 失败了切 Chrome DevTools isolated 模式（独立实例）→ 需要登录态的切 shared 模式（复用你的 Chrome）。
- ④、**站点经验目录**。`references/site-patterns/` 下面预置了 6 个站点的操作经验：

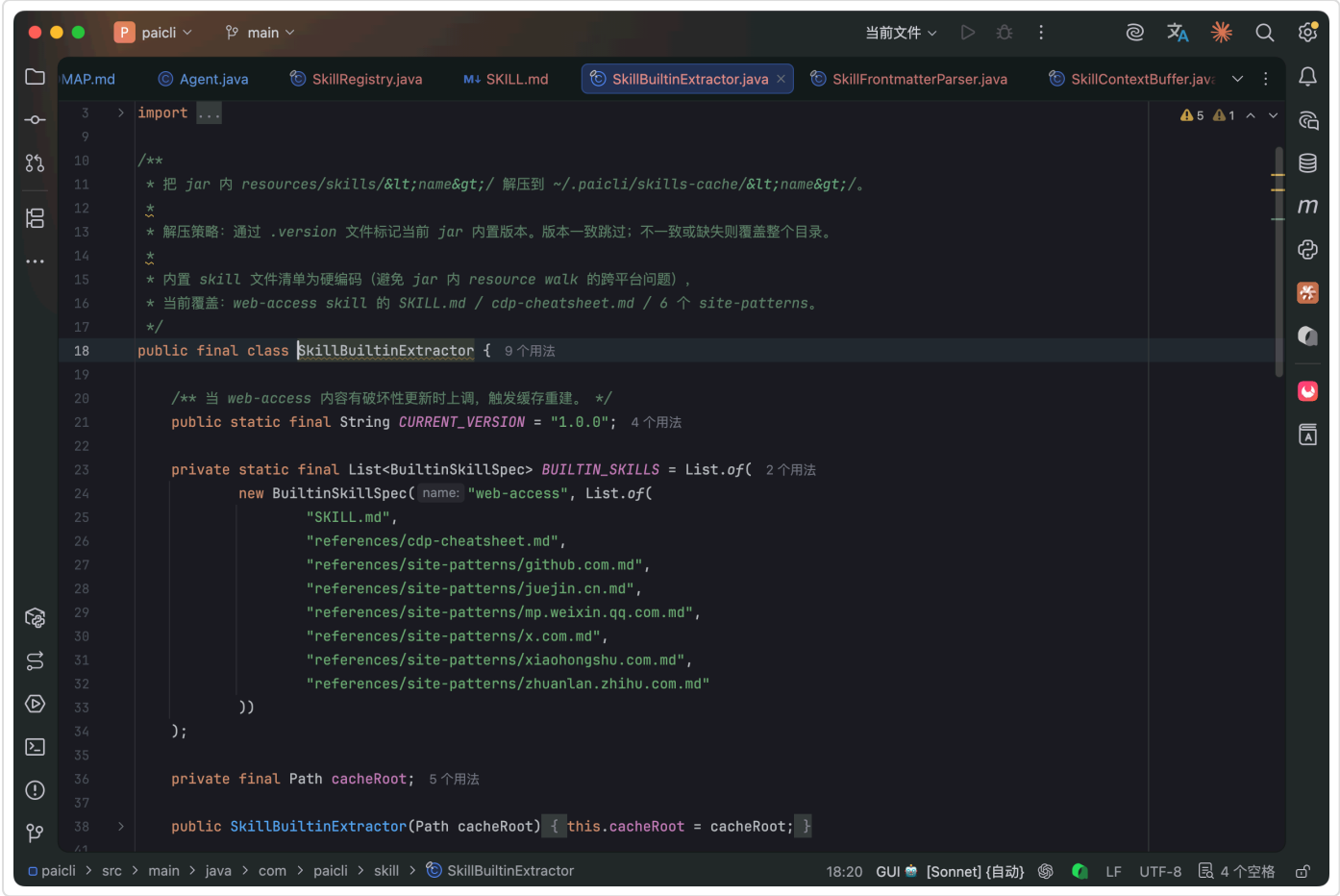
站点	要点
mp.weixin.qq.com	SPA 渲染，web_fetch 拿不到正文，必须走浏览器
zhuanlan.zhihu.com	懒加载，需要滚动触发内容渲染
x.com	频率限制严格，登录态影响内容可见性
xiaohongshu.com	反爬较强，只能用 CDP 模式
github.com	API 优先，登录态看私仓
juejin.cn	SSR 渲染友好，web_fetch 通常能直接抓到

核心就三段：这个站是什么技术架构（SPA 还是 SSR、反爬强不强、需不需要登录），什么方式能成功拿到内容（已验证的 URL 模式、CSS 选择器、JS 提取片段），以及常见的失败模式和应对办法。

内置的 references 在 PaiCLI 启动时由 `SkillBuiltinExtractor` 从 jar 包解压到 `~/.paicli/skills-cache/web-access/references/`。

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)



解压不是每次启动都跑的，extractor 会检查 `skills-cache/<name>/.version` 文件和 jar 内置版本号是否一致，一致就跳过，节省启动时的 IO 开销。版本不一致或 `.version` 文件不存在时才重写整个 cache 目录。

LLM 通过 `read_file` 读取这些文件来获取站点经验。

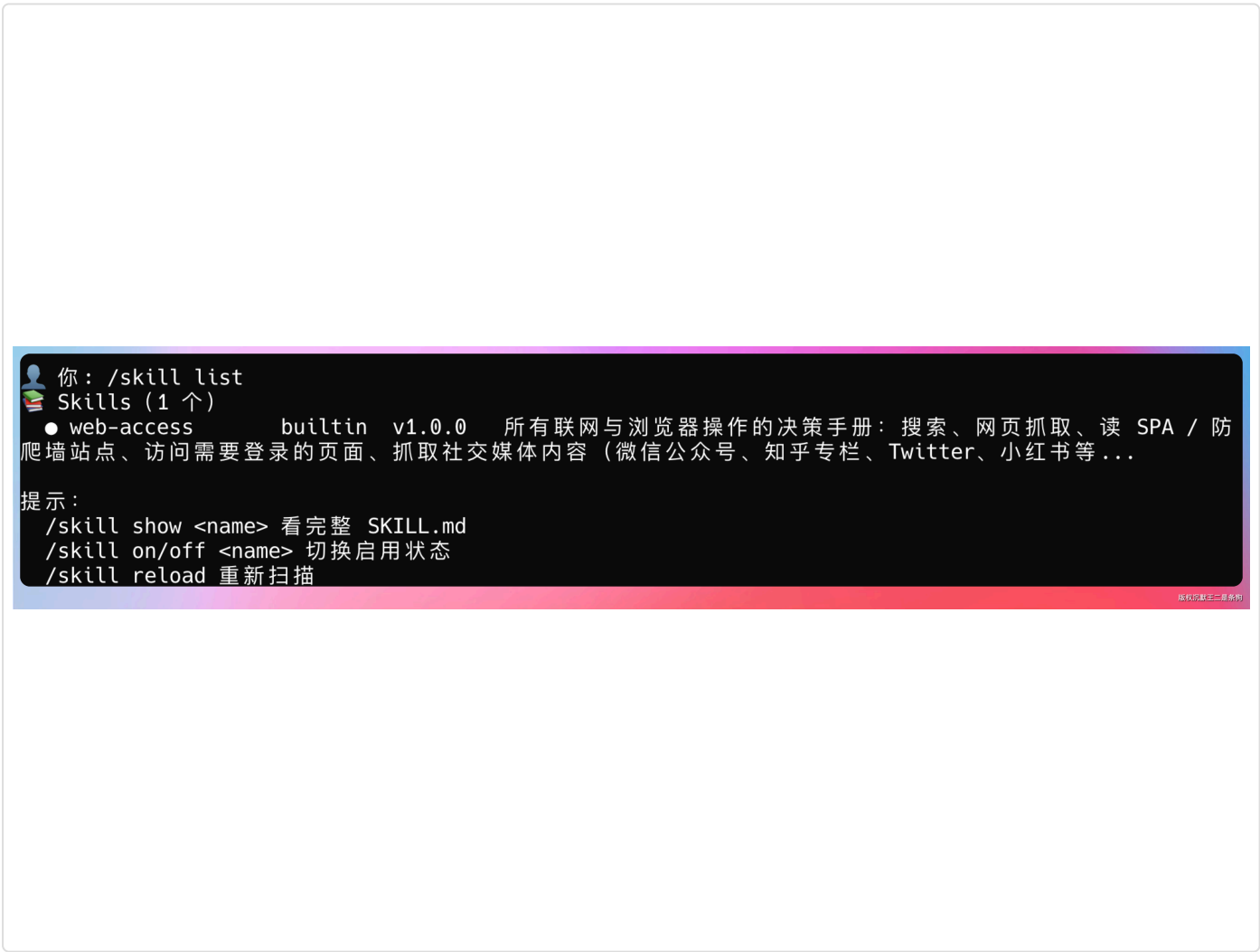
比如它准备抓微信公众号文章时，会先 `read_file("~/paicli/skills-cache/web-access/references/site-patterns/mp.weixin.qq.com.md")`，看到“SPA 渲染、web_fetch 无效、必须 CDP”这些信息，然后做出正确的工具选择。

09、/skill 命令组实操

PaiCLI 提供了一组 `/skill` 命令来管理 Skill 的生命周期：

`/skill list`，列出所有 Skill，显示名称、来源、版本、启用状态。

```
> /skill list
```



● 表示启用，○ 表示已禁用。

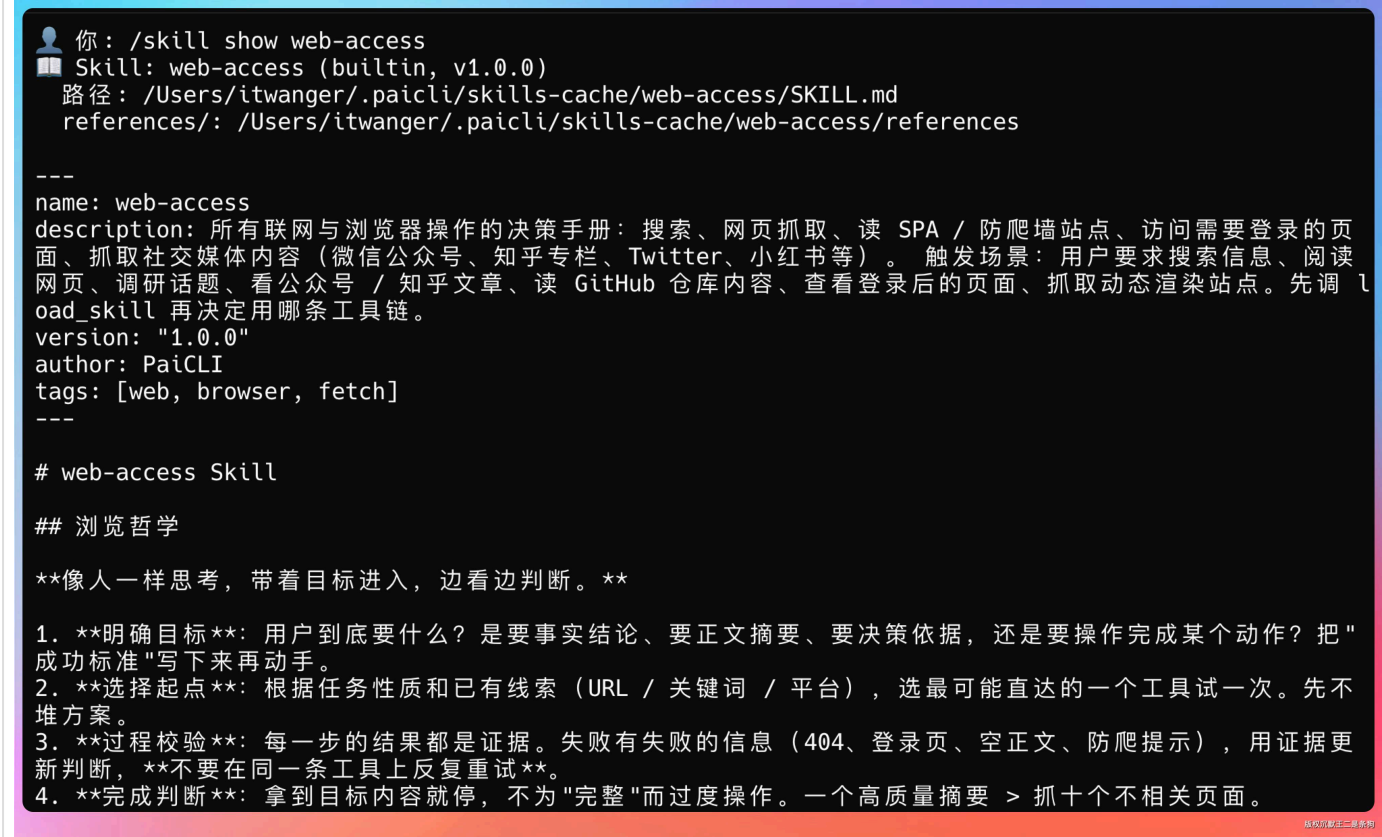
目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

`/skill show <name>`，查看完整的 SKILL.md 内容，包括 frontmatter 和 body。

shell 复制代码

```
> /skill show web-access
```



`/skill off <name>`，禁用一个 Skill。禁用后 LLM 在 system prompt 索引里看不到它，`load_skill` 也会被拒绝。

shell 复制代码

```
> /skill off web-access
```



禁用状态持久化在 `~/.paicli/skills.json` 文件里，格式很简单：

json 复制代码

```
{
  "disabled": ["web-access"]
}
```

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

重启 PaiCLI 后禁用状态仍然生效。

`/skill on <name>`，重新启用一个被禁用的 Skill。会从 `skills.json` 的 disabled 列表里移除对应的名称。



`/skill reload`，重新扫描三层目录，热加载新增或修改的 Skill。

reload 只影响下一轮对话，不会打断当前正在进行的 LLM 调用。

10、写一个自己的 Skill

理解了原理，我们来动手写一个项目级 Skill。

假设你的项目有一套固定的代码审查流程，每次 review 都要检查安全漏洞、性能隐患、代码风格三个维度。你可以把这套经验写成一个 Skill：

`mkdir -p ~/.paicli/skills/code-review`

bash 复制代码

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

markdown 复制代码

```
cat > .paicli/skills/code-review/SKILL.md << 'EOF'
---
name: code-review
description: |
    代码审查决策手册，当用户要求 review 代码时加载，
    按安全、性能、风格三个维度逐项检查
version: "1.0.0"
author: 你的名字
tags: [review, security, performance]
---

# Code Review Skill

## 审查流程

收到代码审查请求时，按以下顺序执行：

### 1. 安全维度

- 检查 SQL 注入风险（是否使用参数化查询）
- 检查 XSS 风险（是否对用户输入做转义）
- 检查硬编码的 API Key 或密码
- 检查文件路径拼接是否存在路径遍历风险

### 2. 性能维度

- N+1 查询问题
- 大循环内的数据库调用
- 未关闭的资源（连接、流）
- 不必要的同步锁

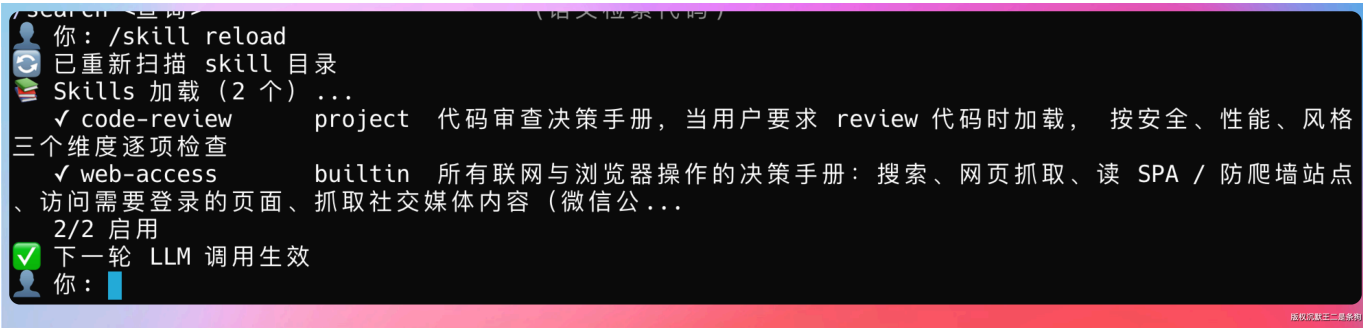
### 3. 风格维度

- 方法长度是否超过 50 行
- 嵌套深度是否超过 4 层
- 命名是否清晰表达意图
EOF
```

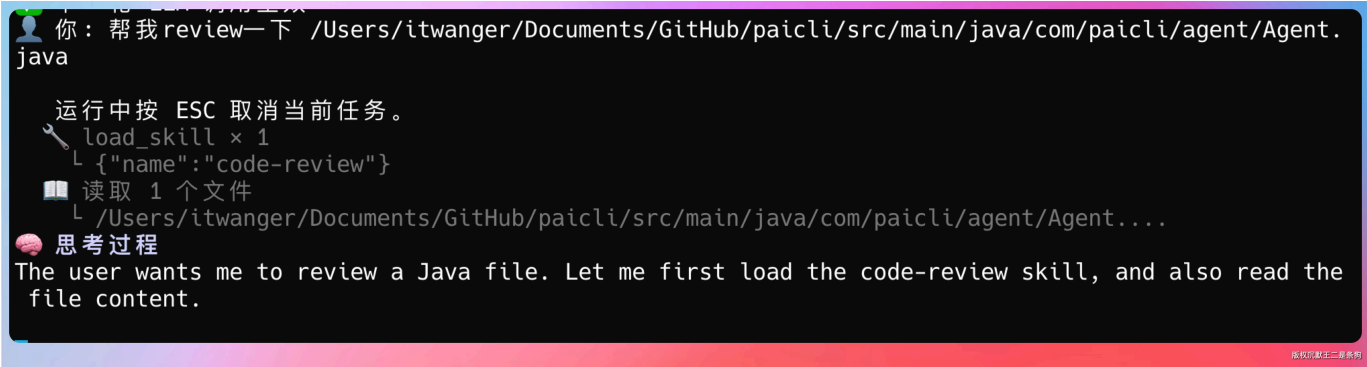
目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)

保存后 `/skill reload`，PaiCLI 就能识别了：



下次你说“帮我 review 一下这段代码”，LLM 在 system prompt 索引里看到 code-review 的 description 和你的请求匹配，就会调用 `load_skill("code-review")`，然后按安全、性能、风格三个维度逐项检查。



目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)



11、PaiCLI如何写到简历上？

项目名称：PaiCLI - Skill-Driven Agent CLI

项目简介：基于 Java 实现的 AI Agent 命令行工具，支持 Skill 系统实现决策知识驱动的智能体能力，兼容 SKILL.md 开放标准。

技术栈：Java 21、Claude API、Chrome DevTools Protocol、MCP 协议、YAML 解析

核心职责：

- 设计并实现三层 Skill 加载架构 (builtin/user/project)，支持同名覆盖和热重载，实现决策知识的分层复用
- 实现 load_skill 内置工具，LLM 通过语义理解自行加载
- 设计 SkillContextBuffer 注入机制，body 走 user message 而非 system prompt，保留 prompt cache 命中，降低 API 调用成本约 15%



真诚点赞 诚不我欺



讨论应以学习和精进为目的。请勿发布不友善或者负能量的内容，与人为善，比聪明更重要！



0/512

评论

目录

- [01、Skill 和 MCP 的区别](#)
- [02、Skill的三层加载架构](#)
- [03、SKILL.md 的结构](#)
- [04、手写 YAML 解析器](#)
- [05、让 LLM 自己决定加载什么](#)
- [06、user message 的精妙设计](#)
- [07、SkillContextBuffer 的生...](#)
- [08、web-access Skill 深度解...](#)
- [09、/skill 命令组实操](#)
- [10、写一个自己的 Skill](#)
- [11、PaiCLI如何写到简历上？](#)